



PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013
Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064



FOOD DELIVERY WEB APPLICATION

AN INTERNSHIP REPORT

Submitted by

THRUPTHI D R- 20231CSE3063

Under the guidance of,

Ms. KALPANA K
BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING



PRESIDENCY UNIVERSITY

BENGALURU

APRIL 2026

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

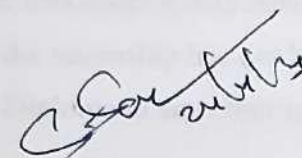
Certified that this report "**Food Delivery Web Application**" is a bonafide work of **Thrupthi D R (20231CSE3063)** who has successfully completed an internship as an AI Software Developer Intern at Tap Academy and carried out the project work as part of the internship. This report is submitted for partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in **Computer Science and Engineering** during 2025-2026.


Ms. Kalpana K

Internship Guide

PSCS

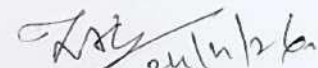
Presidency University


Mr. Sakthi S

Program Internship Coordinator

PSCS

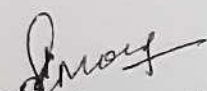
Presidency University


Dr. Md Ziaur Rahaman

School Internship Coordinator

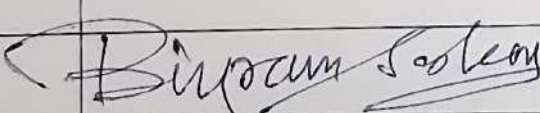
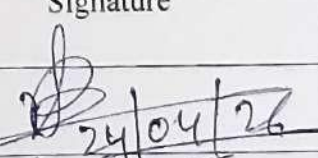
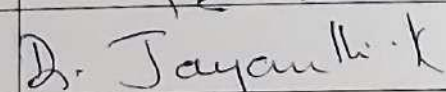
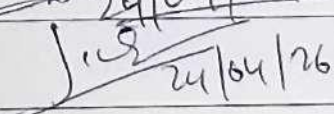
PSCS

Presidency University


Dr. Asif Mohammed
Head of the Department
PSCS
Presidency University


Dr. Duraipandian N
Dean
PSCS / PSIS
Presidency University

Name and Signature of the Examiners

Sl.No	Name of the Examiner	Signature
1	 Birupam Suleen	 24/04/26
2	 Dr. Jayanthi K	 24/04/26

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

I am a student of final year B.Tech in COMPUTER SCIENCE AND ENGINEERING, at Presidency University, Bengaluru, named **THRUPTHI D R**, hereby declare that the internship work titled "**Food Delivery Web Application**" has been independently carried out by me and submitted in partial fulfillment for the award of the degree of B.Tech in COMPUTER SCIENCE ENGINEERING during the academic year of 2025-2026. Further, the matter embodied in the internship has not been submitted previously by anybody for the award of any Degree or Diploma to any other institution.

Thrupthi D R

USN: 20231CSE3063

Signature

Thrupthi D R

PLACE: BENGALURU

DATE: 24 - April 2026

INTERNSHIP COMPLETION CERTIFICATE



+91 8123464489

enquiry@thetapacademy.com

Head Office: TAP Edtech
#3, BTM 2nd Stage, Opp Bharat Petroleum,
Bengaluru 560076
UDYAM-KR-03-04 47516



Date:19/12/2025

To Whomsoever It May Concern

We are delighted to confirm that **Thrupthi D R** from **Presidency University** has been offered the role of **AI Software Developer Intern** at Tap Academy.

Please find the details below:

INTERNSHIP DETAILS	
Position	AI Software Developer Intern
Duration	6 months
Commencement Date	January 5th, 2026
Attendance	Offline (In-person)

During this internship, the student will gain hands-on experience working on a variety of Projects, utilizing technologies such as Core Java, Advanced Java, MySQL, and various front-end technologies. This program is designed to provide practical exposure and enhance skills in a professional environment using AI Tools.

We look forward to welcoming you to Tap Academy and supporting your journey with us.



Thanks & Regards,
Rohit Ravinder
Founder & CEO
Tap EdTech Pvt Ltd

www.thetapacademy.com

ACKNOWLEDGEMENTS

For completing this internship work, I have received the support and the guidance from many people whom I would like to mention with deep sense of gratitude and indebtedness. I extend my gratitude to our beloved **Chancellor, Pro-Vice Chancellor, and Registrar** for their support and encouragement in completion of the internship.

I would like to sincerely thank my internal guide **Ms. Kalpana K, Assistant Professor**, Presidency School of Computer Science and Engineering, Presidency University, for her moral support, motivation, timely guidance and encouragement provided to me during the period of internship work.

I am also thankful to **Dr. Asif Mohamed H B, Professor, Head of the Department, Presidency School of Computer Science and Engineering** Presidency University, for his mentorship and encouragement.

I express my cordial thanks to **Dr. Duraipandian N**, Dean PSCS & PSIS and the Management of Presidency University for providing the required facilities and intellectually stimulating environment that aided in the completion of my internship work.

I am grateful to **Dr. Md Ziaur Rahaman**, Internship Coordinator and **Mr. Sakthi S**, Program Internship Coordinator, Presidency School of Computer Science and Engineering, for facilitating problem statements, coordinating reviews, monitoring progress, and providing their valuable support and guidance.

I am also grateful to Teaching and Non-Teaching staff of Presidency School of Computer Science and Engineering and also staff from other departments who have extended their valuable help and cooperation.

Thrupthi D R

ABSTRACT

The food delivery industry has grown rapidly with the increasing use of the internet, smartphones, and changing customer preferences. Online platforms have made food ordering more convenient compared to traditional methods such as phone calls or visiting restaurants. This report presents the development of a Food Delivery Web Application carried out as part of a training-based internship at Tap Academy. The main objective of this project is to understand and implement the core concepts of building a web-based application using full-stack Java technologies. The application is developed using HTML, CSS, and JavaScript for the frontend, and Java Servlets and JSP for the backend, following the Model–View–Controller (MVC) architecture to ensure proper separation of concerns. MySQL is used as the database, Apache Tomcat as the server, and JDBC for database connectivity.

The application includes basic functionalities such as user registration, login, browsing restaurants, viewing menu items, managing a cart using sessions, and placing orders, along with an admin module for managing menu details and orders. The primary focus of this project is on learning key concepts such as session management, database interaction, and structured design using the DAO pattern, rather than building a fully commercial system. Testing was conducted to verify module functionality, and the system performed correctly for normal operations. Overall, this project helped in gaining practical knowledge of web development and understanding how real-world applications are structured. It also provides scope for future enhancements such as payment integration, delivery tracking, and mobile application support. Additionally, the project helped in understanding the complete lifecycle of web application development, from design to deployment. It improved practical skills in integrating frontend and backend components effectively. The experience gained during this internship provides a strong foundation for developing more complex and scalable applications in the future. It also enhances confidence in working with real-time development tools and technologies.

TABLE OF CONTENTS

Sl. No.	Title	Page No.
	Declaration	iii
	Acknowledgement	i
	Abstract	ii
	List of Figures	v
	List of Tables	vi
	Abbreviations	vii
1.	Introduction 1.1 Background 1.2 Statistics of project 1.3 Prior existing technologies 1.4 Proposed approach 1.5 Objectives 1.6 SDGs 1.7 Overview of project report	1
2.	Literature review	4
3.	Methodology	9
4.	Project management 4.1 Project timeline 4.2 Risk analysis 4.3 Project budget	14
5.	Analysis and Design 5.1 Requirements 5.2 Block Diagram 5.3 System Flow Chart 5.4 Choosing devices 5.5 Designing units 5.6 Standards 5.7 Mapping with IoTWF reference model layers	19

	5.8 Domain model specification 5.9 Communication model 5.10 IoT deployment level 5.11 Functional view 5.12 Mapping IoT deployment level with functional view 5.13 Operational view 5.14 Other Design	
6.	Hardware, Software and Simulation 6.1 Hardware 6.2 Software development tools 6.3 Software code 6.4 Simulation	30
7.	Evaluation and Results 7.1 Test points 7.2 Test plan 7.3 Test result 7.4 Insights	37
8.	Social, Legal, Ethical, Sustainability and Safety Aspects 8.1 Social aspects 8.2 Legal aspects 8.3 Ethical aspects 8.4 Sustainability aspects 8.5 Safety aspects	45
9.	Conclusion	47
	References	48
	Base Paper	
	Appendix	49

LIST OF FIGURES

Figure	Figure Caption	Page No
Figure 3.1	Agile Methodology Model	9
Figure 5.1	Use Case Diagram of Food Delivery System	20
Figure 5.2	System Architecture of Food Delivery Web Application	21
Figure 5.3	Block Diagram of Food Delivery Web Application	22
Figure 5.4	System Flowchart of Food Delivery Application	23
Figure 5.5	Entity Relationship Diagram	27
Figure C.1	User Login Page	58
Figure C.2	User Registration Page	58
Figure C.3	Restaurant Listing Page	59
Figure C.4	Menu Page	59
Figure C.5	Cart Page	60
Figure C.6	Order Confirmation Page	60

LIST OF TABLES

Sl. No	Table Number	Table Caption	Page No
1	Table 2.1	Summary Table of Literature	8
2	Table 3.1	Mapping of Project Activities to Methodology	12
3	Table 4.1	Project Timeline	14
4	Table 4.2	Identified Risks and Mitigation Strategies	15
5	Table 4.3	Estimated Budget Breakdown	17
6	Table 5.1	Database Schema Description	28
7	Table 6.1	Hardware Requirements	30
8	Table 6.2	Software Requirements	31
9	Table 7.1	Test Cases	38
10	Table 7.2	Test Results	41
11	Table B.1	Users Table Structure	53
12	Table B.2	Restaurant Table Structure	54
13	Table B.3	Menu Table Structure	55
14	Table B.4	Orders Table Structure	56
15	Table B.5	Order Item Table Structure	57

ABBREVIATIONS

Abbreviation	Full Form
URL	Uniform Resource Locator
HTTP	HyperText Transfer Protocol
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
JSP	Java Server Pages
JDBC	Java Database Connectivity
DAO	Data Access Object
MVC	Model View Controller
SQL	Structured Query Language
DBMS	Database Management System
API	Application Programming Interface
UI	User Interface
UX	User Experience
IDE	Integrated Development Environment
JDK	Java Development Kit
AJAX	Asynchronous JavaScript and XML

Chapter 1

INTRODUCTION

1.1 Background of the Project

The process of ordering food has evolved significantly with the advancement of digital technology. In the past, customers relied on visiting restaurants or placing orders through phone calls, which often lacked convenience and transparency. There was no easy way to explore menus or track orders, making the experience less efficient.

With the widespread use of smartphones and internet services, online food delivery platforms have transformed this process. Users can now browse restaurants, view menus with detailed information, and place orders quickly. These platforms also provide real-time updates and improve the overall user experience. During my internship at Tap Academy, this project was developed as part of a learning process to understand how real-world web applications are built. The main objective is to gain practical knowledge of concepts such as session management, database connectivity, and user interface design using full-stack Java technologies.

1.2 Statistics and Need of Online Food Delivery Systems

The online food delivery industry has experienced rapid growth due to increasing internet accessibility and changing consumer habits. In India, especially in urban areas, people prefer online food services because of their convenience and time-saving benefits.

The COVID-19 pandemic further accelerated the adoption of digital platforms, including food delivery services. Even after the pandemic, many users continue to rely on these platforms for daily needs. From an academic perspective, developing a food delivery application provides valuable hands-on experience in understanding system design, user data handling, session management, and responsive interface development.

1.3 Existing Technologies and Systems

Many platforms such as Swiggy, Zomato, Uber Eats, and DoorDash are widely used for food delivery services. These platforms use advanced technologies like real-time tracking, cloud computing, and scalable systems to handle large numbers of users. At a basic level, food delivery systems are built using web technologies. Earlier, developers used PHP and MySQL,

while modern systems use technologies such as Node.js, Python, and Java frameworks. In this project, Java Servlets and JSP are used to understand the core concepts of web development. Apache Tomcat is used as the server, and MySQL is used as the database. These technologies provide a strong foundation for learning how web applications function internally.

1.4 Proposed System

The proposed system is a web-based food delivery application developed as part of internship training. It allows users to register, log in, browse restaurants, view menu items, add items to a cart, and place orders. The system also includes an admin module for managing menu items and viewing orders. The main aim of this project is to understand and implement the concepts of full-stack web development using Java technologies.

The project is motivated by the need to gain practical knowledge and experience in building real-world applications. The system follows the MVC architecture, where Servlets handle the logic, JSP handles the user interface, and DAO classes manage database operations. This application is mainly developed for learning purposes and can be used as a basic prototype. However, it does not include advanced features such as online payment, delivery tracking, or mobile application support.

1.5 Objectives

The main objective of this project is to understand and implement a basic food delivery web application using full-stack Java technologies. The project focuses on applying the MVC architecture for better structure and organization. It also aims to design and manage a relational database using MySQL and perform secure database operations using JDBC and DAO pattern. Another objective is to develop a simple and user-friendly interface using HTML, CSS, and JavaScript. Additionally, the project involves testing the system to ensure correct functionality under different scenarios. Additionally, the project aims to understand the complete workflow of a web application from development to deployment on a server like Apache Tomcat. It also focuses on improving problem-solving and debugging skills while handling real-time issues during development. Furthermore, the project helps in gaining hands-on experience with development tools and understanding best practices followed in real-world applications.

1.6 SDGs Alignment

This project is indirectly related to certain Sustainable Development Goals (SDGs). It supports SDG 8, which focuses on decent work and economic growth, as digital platforms create job opportunities in the food delivery sector. It also relates to SDG 9, which promotes industry, innovation, and infrastructure through the development of technology-based solutions. Additionally, it supports SDG 12 by encouraging digital systems that reduce manual processes and paper usage.

1.7 Overview of Report

This report is organized into multiple chapters, each explaining a different part of the project. Chapter 2 discusses the literature review related to the technologies used. Chapter 3 explains the methodology followed during development. Chapter 4 covers project management aspects such as timeline and risks. Chapter 5 presents system analysis and design. Chapter 6 explains implementation details. Chapter 7 includes testing and results. Chapter 8 discusses social and ethical aspects, and Chapter 9 concludes the report with future enhancements.

Chapter 2

LITERATURE SURVEY

This chapter presents a review of the concepts and technologies used in the development of the Food Delivery Web Application. The focus is on understanding the basic ideas behind web application development, Java-based technologies, database connectivity, and system design patterns. The purpose of this review is not only to study existing concepts but also to support the learning process during the internship. Through this, it becomes easier to understand how real-world web applications are structured and how different components work together.

2.1 Evolution of Web Applications and Online Food Delivery

Web applications have changed significantly over time. In the early stages, websites were static and could only display fixed information. Users could not interact with the system. With the introduction of server-side technologies such as CGI, PHP, ASP, and later JSP, websites became dynamic and capable of responding to user inputs. Over time, web applications started replacing traditional desktop applications in many areas such as banking, shopping, and food services. Online food delivery systems became popular because they allow users to view menus, place orders, and receive updates in real time. In this project, understanding this evolution helps in learning how modern web applications are built. It also gives a clear idea of how user interaction is handled through dynamic web technologies and how systems have improved over time.

2.2 MVC Architecture in Web Development

In this project, the MVC architecture is used to organize the application into separate components for better clarity and maintainability. The Model represents the data and handles business logic, the View is responsible for presenting information to the user, and the Controller manages user requests and controls the application flow.

This separation makes the system easier to understand, maintain, and extend. It also allows different parts of the application to be developed and modified independently without affecting others. In this implementation, Servlets act as controllers, JSP pages are used as the view layer, and Java classes represent the model. Overall, using MVC improves the structure of the application, simplifies debugging, and makes development more organized and efficient.

2.3 Java Servlets and JSP

Java Servlets and JSP are core technologies used for building dynamic web applications. Servlets act as the backend component that handles client requests and generates appropriate responses on the server side. They are responsible for processing business logic and managing the flow of data within the application. JSP, on the other hand, is used to design the user interface by embedding Java code within HTML. When a JSP page is executed, it is internally converted into a Servlet by the server. This allows developers to separate presentation from processing logic. In this project, Servlets and JSP are integrated to manage request handling and user interface rendering effectively, providing a clear understanding of how web applications operate.

2.4 Data Access Object (DAO) Pattern

The Data Access Object (DAO) pattern is used to manage database operations in a structured way. It separates database-related code from the main application logic, making the system easier to maintain, understand, and modify. In this project, DAO classes are created for handling operations such as inserting, updating, and retrieving data from the database. This approach improves code organization and reduces repetition of database-related code. Using the DAO pattern also helps in improving security and maintainability, as all database interactions are controlled in one place.

2.5 JDBC and Database Connectivity

JDBC is a standard API that enables Java applications to interact with relational databases. It provides methods to execute SQL queries, retrieve results, and perform data manipulation operations. In this project, JDBC is used to establish a connection with the MySQL database and handle operations such as inserting user data, fetching menu details, and processing orders. Prepared statements are used to securely pass parameters and prevent SQL injection attacks. This implementation helps in understanding how backend systems communicate with databases and manage application data efficiently. Additionally, JDBC supports different types of drivers that help in connecting Java applications to various databases. It also allows efficient handling of database connections through connection management techniques.

2.6 MySQL as a Relational Database

MySQL is used as the database system in this project due to its reliability, simplicity, and widespread use in web applications. It stores data in structured tables and supports relationships between different entities. In this application, MySQL is used to manage data related to users, restaurants, menu items, and orders. Proper table design and relationships ensure data consistency and efficient retrieval. Working with MySQL in this project provides practical knowledge of database design, query execution, and data management in real-world applications.

2.7 Apache Tomcat as a Servlet Container

Apache Tomcat is used as the server to run the application. It provides an environment where Servlets and JSP pages can be executed properly. Tomcat handles request processing, session management, and application deployment. It acts as a bridge between the user and the backend system. In this project, Tomcat is used to understand how web applications are deployed and how server-side processing works. Additionally, Tomcat supports easy configuration and deployment of web applications using WAR files. It also helps developers test applications locally before deploying them in a production environment.

.

2.8 Security Considerations in Web Applications

Security is an important part of web application development. Common issues include unauthorized access, SQL injection, and invalid user inputs. In this project, basic security measures are implemented such as session management for user authentication, input validation, and the use of prepared statements.

These methods help in protecting user data and ensuring secure communication between components. Understanding these security concepts is important for developing reliable applications. Additionally, proper session handling helps in maintaining secure user login states and prevents unauthorized access to protected pages. Basic validation techniques ensure that incorrect or harmful data is not processed by the system. These practices provide a foundation for implementing more advanced security features in future applications.

2.9 Agile Methodology in Academic Projects

Agile methodology is a flexible approach to software development where work is completed in small stages. It allows continuous improvement and helps in managing changes easily. During this internship, the project was developed step by step. Different modules such as login, menu display, cart, and order processing were implemented gradually.

This approach made it easier to understand each part of the system clearly. Using this method helped in better learning and effective development of the project. Additionally, Agile allows quick feedback and improvements during each development stage. It helps in identifying issues early and making necessary changes without affecting the overall system. This approach improves both learning and productivity in academic projects.

2.10 Existing Food Delivery System Research

Existing food delivery systems provide features such as restaurant listing, menu display, and order processing. These systems highlight the importance of managing user data properly and providing a smooth user experience. In this project, similar features are implemented to understand how such systems work. The focus is not on building a large-scale application but on learning the core concepts of web development.

This helps in gaining practical knowledge and prepares for developing more advanced systems in the future. Additionally, real-world food delivery platforms include advanced features such as live order tracking, online payment integration, and personalized recommendations based on user preferences. These systems are designed to handle a large number of users efficiently and ensure fast performance. Studying these systems helps in understanding how scalability, performance, and user experience are maintained in real applications. This knowledge can be useful for enhancing the current project with more advanced features in the future.

2.1 Summary Table of Literature

Author(s)	Year	Focus Area	Key Learning / Relevance to Project
Tim Berners-Lee	1990	World Wide Web	Foundation for understanding how web applications work
Gamma et al	1994	Design Patterns	MVC concept used to structure the application
Alur, Crupi & Malks	2001	J2EE Patterns	DAO pattern used for database interaction
Beck et al.	2001	Agile Methodology	Iterative development followed during internship
Bergsten	2003	JSP Technology	Helps in building dynamic web pages
Moodie	2007	Apache Tomcat	Understanding server and deployment process
Horstmann & Cornell	2011	Java Development	Basics of Servlets and backend development
Schwartz et al.	2012	MySQL	Database design and data handling concepts
White & Fisher	2016	JDBC	Database connectivity using Java
Various Sources	2019	Online Food Delivery	Understanding workflow of online food ordering
Kumar & Sharma	2020	Web Applications	Handling multiple users and data consistency
Patel et al.	2021	System Design	Basic structure of scalable applications

Chapter 3

METHODOLOGY

This chapter explains the methodology followed for developing the Food Delivery Web Application during the internship at Tap Academy. Since this was a training-based internship, the main focus was not only on building a working system but also on understanding how different components of a web application are designed and implemented. A structured yet flexible approach was followed to balance both learning and development. The project mainly follows the Agile Software Development Life Cycle (SDLC), along with some elements of the traditional approach for better planning and documentation.

3.1 Methodology Overview

Agile methodology was chosen for this project because it allows development in small steps and helps in understanding the system gradually. In a learning environment, requirements are not always clear at the beginning, and they become better understood during development. For example, user interface flow and database relationships were refined while implementing different modules.

Another reason for choosing Agile was the limited duration of the internship. Instead of building the entire system at once, the project was divided into smaller parts so that each part could be completed, tested, and improved step by step. The work was organized into short development cycles, where progress was reviewed regularly, and necessary changes were made. This approach helped in maintaining both flexibility and structure throughout the project.



Figure 3.1: Agile Methodology Model

3.2 Requirements Gathering and Analysis

The initial phase of the project focused on understanding the requirements of the system. Two main types of users were identified: customers and administrators. Customers use the application to browse restaurants, view menu items, and place orders, while administrators manage menu details and view orders.

The requirements were identified through discussions with the internship mentor, observation of existing platforms such as food delivery applications, and studying basic concepts related to web applications. These requirements were then divided into functional and non-functional categories. Functional requirements include features such as user registration, login, restaurant browsing, cart management, and order placement. Non-functional requirements include performance, security, and usability aspects. This phase helped in creating a clear understanding of what the system should do and how different modules are connected.

3.3 System Design

After identifying the requirements, the next step was to design the system. The application was designed using the Model–View–Controller (MVC) architecture, which helps in separating the user interface, business logic, and data handling. This makes the system easier to understand and maintain.

The database design was created using MySQL, where tables such as user, restaurant, menu item, cart, and order were defined. Relationships between these tables were established to ensure proper data management. The database was structured carefully to avoid redundancy and maintain consistency. In addition, a layered structure was followed in the project, where DAO classes were used to handle database operations, Servlets were used to control application flow, and JSP pages were used to display data to the user. Basic user interface layouts were also planned before implementation to ensure smooth navigation.

3.4 Development Process

The development of the application was carried out in multiple stages. Initially, the project setup was completed in Eclipse, and database connectivity was established using MySQL. The first module implemented was the user authentication system, which included user registration, login, and session management. After that, the restaurant and menu module was developed, allowing users to view available restaurants and their menu items. The cart management module was then implemented, where users could add items, update quantities,

and remove items from the cart. Finally, the order processing and admin module were developed, enabling users to place orders and administrators to manage menu items and view order details. This step-by-step development helped in understanding each module clearly and ensured that all parts of the application were properly connected.

3.5 Testing

Testing was carried out to ensure that all parts of the application work correctly. Different types of testing were performed, including checking individual components, verifying interaction between modules, and testing the complete system. Both valid and invalid input cases were tested to observe system behavior. For example, correct login credentials, incorrect passwords, and empty input fields were tested to ensure proper validation. The results were checked through both the user interface and the database to confirm that data was processed correctly. Testing helped in identifying and fixing errors, improving system reliability, and ensuring that the application performs as expected.

3.6 Documentation and Deployment

The final phase of the project involved documentation and deployment. All project details, including design, implementation, and testing, were documented properly. The code was reviewed, and necessary improvements were made to enhance readability and structure. The application was deployed on the Apache Tomcat server and tested in a browser to ensure proper functioning. Basic setup instructions were also prepared so that the application can be executed easily. This phase ensured that the project is complete, organized, and ready for submission.

Table 3.1 Mapping of Project Activities to Methodology

Sprint	Duration	SDLC Phase	Key Deliverables
Sprint 0	Week 1–2	Planning & Design	Requirement analysis, database schema design, architecture diagram, basic UI wireframes
Sprint 1	Week 3–4	Development	User authentication module, database connectivity setup, login and registration pages
Sprint 2	Week 5–6	Development	Restaurant listing feature, menu display, implementation of DAO classes
Sprint 3	Week 7–8	Development	Cart management system, updating cart items, dynamic updates using AJAX
Sprint 4	Week 9–10	Development	Order placement functionality, admin panel development, order status handling
Sprint 5	Week 11–12	Testing & Documentation	Test case execution, bug fixing, report preparation, application deployment

Table 3.1 presents the mapping between project activities and the adopted Agile-based SDLC methodology. It clearly illustrates how each phase of the project was executed through structured sprints over the internship duration. As shown in Table 3.1, the project began with Sprint 0, which corresponds to the Planning and Design phase. During this phase, requirement analysis, database schema design, system architecture (MVC), and UI wireframes were prepared to establish a strong foundation.

The subsequent sprints (Sprint 1 to Sprint 4) fall under the Development phase. Each sprint focused on implementing specific modules of the system. This includes user authentication, restaurant listing, menu display, cart management, and order processing functionalities. The use of technologies like JSP, Servlets, JDBC, and DAO pattern ensured modular and maintainable development. Finally, Sprint 5 represents the Testing and Documentation phase, where test cases were executed, bugs were fixed, and the complete project report was prepared. The application was also deployed on the server during this stage. This mapping demonstrates a clear alignment between Agile practices and SDLC phases, ensuring systematic development, continuous improvement, and timely delivery of project components.

Chapter 4

PROJECT MANAGEMENT

Effective project management played an important role in completing the Food Delivery Web Application within the given internship timeline while maintaining the required quality standards. This chapter describes the project timeline, risk analysis, and budget estimation.

4.1 Timeline

The project was completed over a twelve-week internship period at Tap Academy. The following table presents a Gantt-style breakdown of the project phases and their corresponding weeks.

Table 4.1: Project Timeline

Phase / Activity	Wk 1	Wk 2	Wk 3	Wk 4	Wk 5	Wk 6	Wk 7	Wk 8	Wk 9	Wk 10	Wk 11	Wk 12
Requirement Analysis	✓	✓										
System Design		✓	✓									
DB Setup & Config			✓									
User Auth Module			✓	✓								
Restaurant & Menu					✓	✓						
Cart Management							✓	✓				
Order & Admin Panel									✓	✓		
Testing										✓	✓	
Documentation											✓	✓
Report Writing											✓	✓

This Table 4.1 represents the project timeline for the 12-week internship using a Gantt-style format. It shows how different activities of the project were planned and completed across each week. The table indicates that the project started with requirement analysis in Week 1 and Week 2, followed by system design in Week 2 and Week 3. After that, database setup was done in Week 3, and then the user authentication module was developed during Week 3 and Week 4. In the next phase, restaurant and menu features were implemented in Week 5 and Week 6, followed by cart management in Week 7 and Week 8. The order processing and admin panel were developed in Week 9 and Week 10. Finally, testing was carried out in Week 10 and Week 11, and documentation and report writing were completed in Week 11 and Week 12. Overall, this table shows that the project was developed in a step-by-step manner, where some phases overlapped to save time and improve efficiency. It reflects a structured approach to completing the project within the given internship duration.

4.2 Risk Analysis

Table 4.2: Identified Risks and Mitigation Strategies

Risk	Category	Likelihood	Impact	Mitigation Strategy
Unfamiliarity with JSP/Servlet lifecycle	Technical	High	High	Self-learning, mentor guidance, and use of online resources
Database schema changes during development	Technical	Medium	Medium	Apply normalization early and use DAO pattern to manage changes
Tomcat configuration errors	Technical	Medium	Low	Follow proper documentation and test deployment at early stages
Scope creep (adding extra features)	Time	Medium	High	Define MVP clearly and postpone additional features

Code loss without version control	Resource	Low	High	Use Git for version control and maintain regular backups
Insufficient testing time	Time	Medium	Medium	Perform testing in each sprint, reserve final weeks
SQL injection vulnerabilities	Security	Medium	High	Use Prepared Statements and validate all user inputs

This table represents the possible risks identified during the project development along with their category, likelihood, impact, and the strategies used to handle them. The table shows that different types of risks were considered such as technical risks, time-related risks, resource risks, and security risks. For example, unfamiliarity with JSP and Servlets was a high-risk technical issue, which was handled through self-learning and mentor guidance. Similarly, database changes during development were managed by proper design using normalization and the DAO pattern. Other risks like Tomcat configuration errors were handled by following proper documentation and early testing.

Time-related risks such as scope creep and insufficient testing time were controlled by defining a clear project scope (MVP) and performing testing in each phase. Resource risks like code loss were avoided by using Git and maintaining backups. Security risks such as SQL injection were addressed by using prepared statements and validating user inputs. Overall, this table shows that risks were identified early and proper strategies were applied to reduce their impact and ensure smooth project development.

4.3 Budget

Since this is an academic internship project, the overall cost is minimal as most of the tools used are free and open-source. However, an estimated budget is provided to understand the potential cost if the system is deployed in a real-world environment.

Table 4.3: Estimated Budget Breakdown

Resource	Description	Cost (INR / Month)	Notes
Development Machine	Laptop (i5, 8GB RAM)	Existing Asset	No additional cost for internship
Eclipse IDE	Development environment	Free	Eclipse IDE for Java EE Developers
Apache Tomcat	Web server	Free	Apache Software Foundation
MySQL	Database	Free	Oracle MySQL Community Server
JDK (Java 11)	Development kit	Free	OpenJDK distribution
Domain Name	For production deployment	₹ 1,000 / year	Optional for academic project
VPS Hosting	Cloud hosting	₹ 2,000 / month	Not required for academic submission
SSL Certificate	For HTTPS	Free	Required for production
Total (Academic Cost)	—	₹ 0	All tools are free and open-source
Total (Production cost)	—	₹ 3,000 / month	Estimated minimum for live deployment

This table shows the estimated cost required for developing and deploying the project. Since this is an academic internship project, most of the tools used such as Eclipse IDE, Apache Tomcat, MySQL, and JDK are free and open-source, so there is no actual cost involved during development. The table also includes some resources like a development machine (laptop), which is already available, so no additional expense is required.

However, to understand real-world deployment, some optional costs are mentioned such as domain name, VPS hosting, and SSL certificate. If the application is deployed in a real environment, the main cost would be hosting (around ₹2000/month) and a domain name (around ₹1000/year). The SSL certificate is free but important for secure communication. Overall, this table shows that the project is cost-effective for academic purposes (₹0 cost), but in a real-world scenario, it may require a minimum monthly cost for hosting and deployment.

Chapter 5

ANALYSIS AND DESIGN

5.1 Functional and Non-Functional Requirements

5.1.1 Functional Requirements

Functional requirements describe the specific operations and capabilities that the system must perform. In the Food Delivery Web Application, users are provided with the ability to register by entering their name, email address, password, and contact number, with the system ensuring that the email is unique before account creation. Registered users can log in using valid credentials, after which a session is created to maintain authentication throughout their interaction, and they can log out to terminate the session securely. Once logged in, users can browse a list of available restaurants, each displaying essential details such as the restaurant name, cuisine type, and description.

Upon selecting a restaurant, the system presents its menu, including item names, descriptions, and prices. Users can add items to a shopping cart, view the cart with updated totals, modify item quantities, or remove items as needed. The system also supports order placement, where users proceed to checkout, confirm their delivery details, and place orders, after which a confirmation message is displayed. Additionally, administrators can log in through the same interface using admin credentials, gaining access to a dedicated panel where they can manage menu items by adding, updating, or deleting them, as well as view all customer orders along with their detailed information.

5.1.2 Non-Functional Requirements

Non-functional requirements define the quality attributes and performance standards of the system. The application is designed to ensure efficient performance, where pages load within a few seconds under normal conditions and database operations are executed promptly. Security is maintained by using Prepared Statements to prevent SQL injection attacks, along with session management handled by the Servlet container to ensure secure user authentication.

Although password hashing is not implemented in the current prototype, it is identified as an important enhancement for future versions. The system is developed with usability in mind, offering an intuitive interface that allows even first-time users to navigate and complete tasks easily, supported by clear and meaningful error messages. Maintainability is ensured by

following the MVC architecture and DAO design pattern, enabling modular development and easier updates without affecting other components. The system is also designed with scalability considerations, allowing future expansion with minimal changes. In terms of availability, the application is expected to run reliably on a local server without interruptions during normal usage.

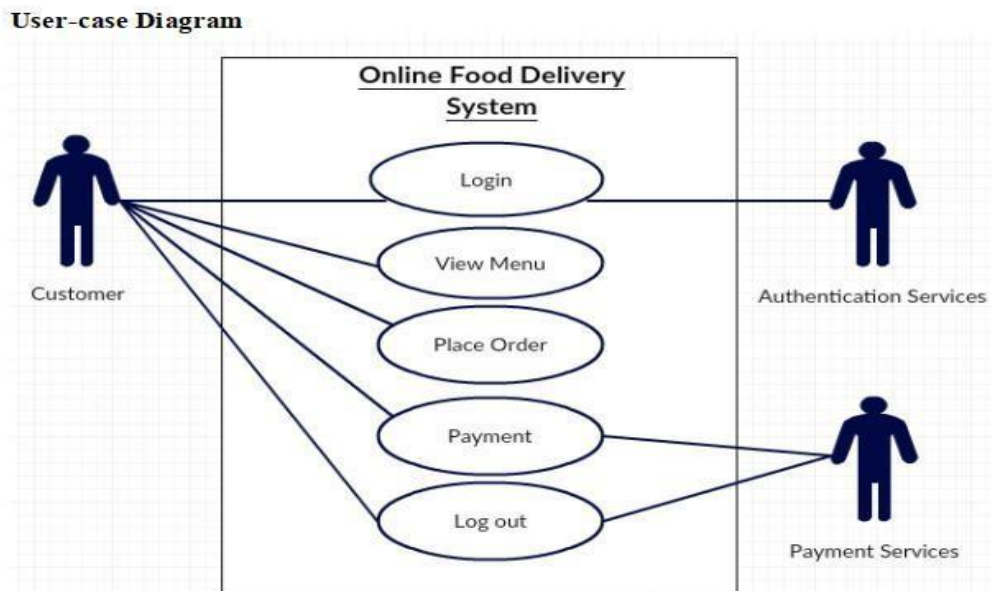


Figure 5.1: Use Case Diagram of Food Delivery System

This use case diagram represents the interaction between users and the system. Customers can register, log in, browse restaurants, view menu items, add items to the cart, and place orders. Administrators can manage menu items and view orders. It provides a high-level overview of system functionality.

5.2 System Architecture – MVC Explanation

The Food Delivery Web Application is built using the Model-View-Controller (MVC) architectural pattern, which ensures a clear separation of concerns within the system. The Model layer consists of Java classes that represent application data such as users, restaurants, menu items, carts, and orders, along with DAO classes that handle database interactions using JDBC. These classes primarily function as data holders without containing business logic. The View layer is implemented using JSP pages, which are responsible for presenting data to the user in a structured and visually understandable format.

These pages access data through request attributes and utilize JSTL tags to simplify dynamic content rendering. The Controller layer is composed of Java Servlets that handle user requests,

process input data, interact with the DAO layer, and determine the appropriate view to display. This structured approach ensures that modifications in one layer do not affect others, thereby improving system maintainability, flexibility, and scalability.

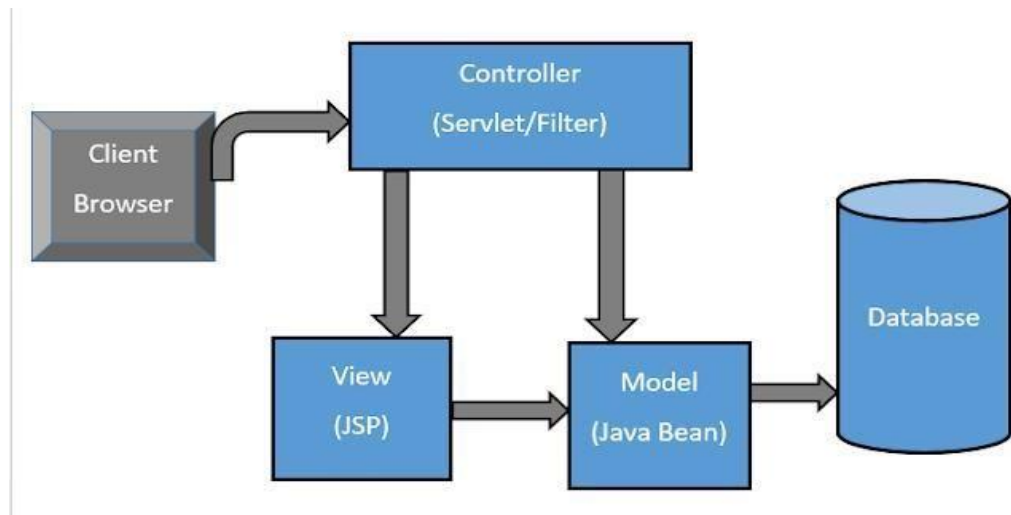


Figure 5.2: System Architecture of Food Delivery Web Application

This diagram illustrates the Model–View–Controller (MVC) architecture used in the Food Delivery Web Application. The client browser sends HTTP requests to the Controller (Servlet/Filter), which processes the request and interacts with the Model (Java Beans and DAO classes) for business logic and database operations. The processed data is then forwarded to the View (JSP), which generates the response and displays it to the user.

5.3 Block Diagram Description

The system architecture follows a layered design approach, beginning with the client browser, which sends HTTP requests and receives responses. These requests are processed by the Apache Tomcat server, which acts as the web server and servlet container. Within the application, the architecture is divided into three main layers: the presentation layer consisting of JSP pages, the control layer consisting of Servlets, and the data layer consisting of DAO classes and model objects. The MySQL database forms the backend storage system, connected via JDBC. When a request is made, it flows from the browser to the servlet, which processes

it, interacts with the database through DAO classes, and forwards the processed data to a JSP page for rendering. The final response is then sent back to the user's browser.

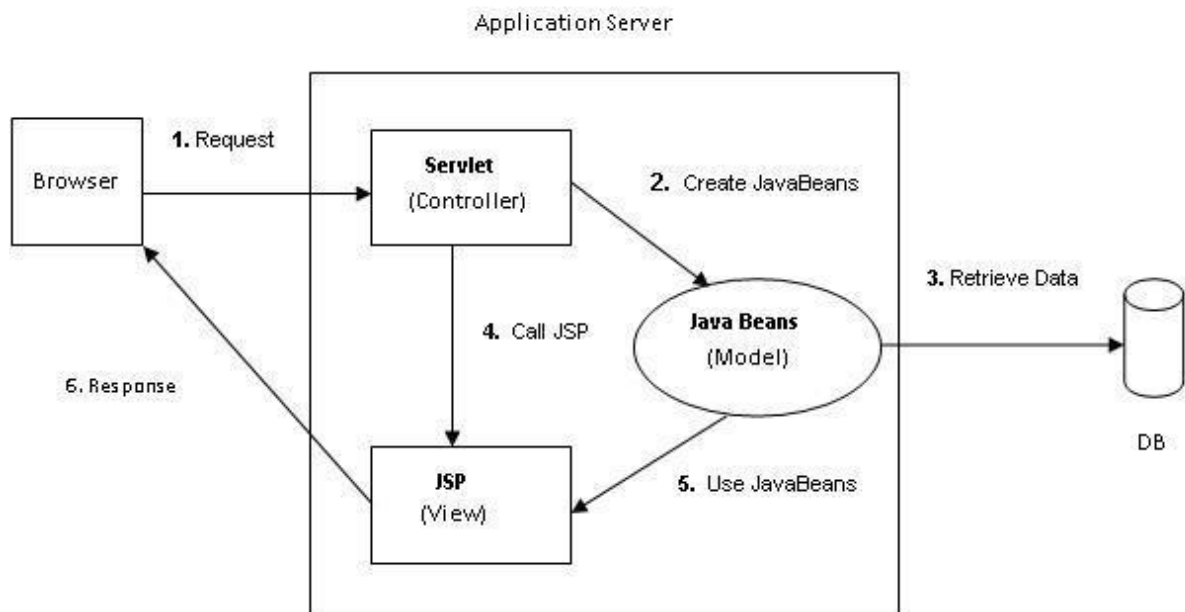


Figure 5.3: Block Diagram of Food Delivery Web Application

This block diagram represents the high-level structure of the Food Delivery Web Application. The client browser sends requests to the server, where Servlets process the logic. The DAO layer interacts with the MySQL database, and the response is returned to the user through JSP pages.

5.4 Flowchart Explanation

The application workflow begins when the user accesses the system through a browser and is presented with the login page. The system checks whether a valid session exists; if not, the user is required to log in by entering valid credentials. Upon successful authentication, the user is redirected to the restaurant listing page, where they can browse available options. Selecting a restaurant displays its menu, allowing users to add items to their cart. The user can review the cart, update quantities, or remove items before proceeding to checkout. During checkout, the user confirms delivery details and places the order. The system then records the order in the database, clears the cart, and displays an order confirmation message. After completing the process, the user may continue browsing or log out of the application.

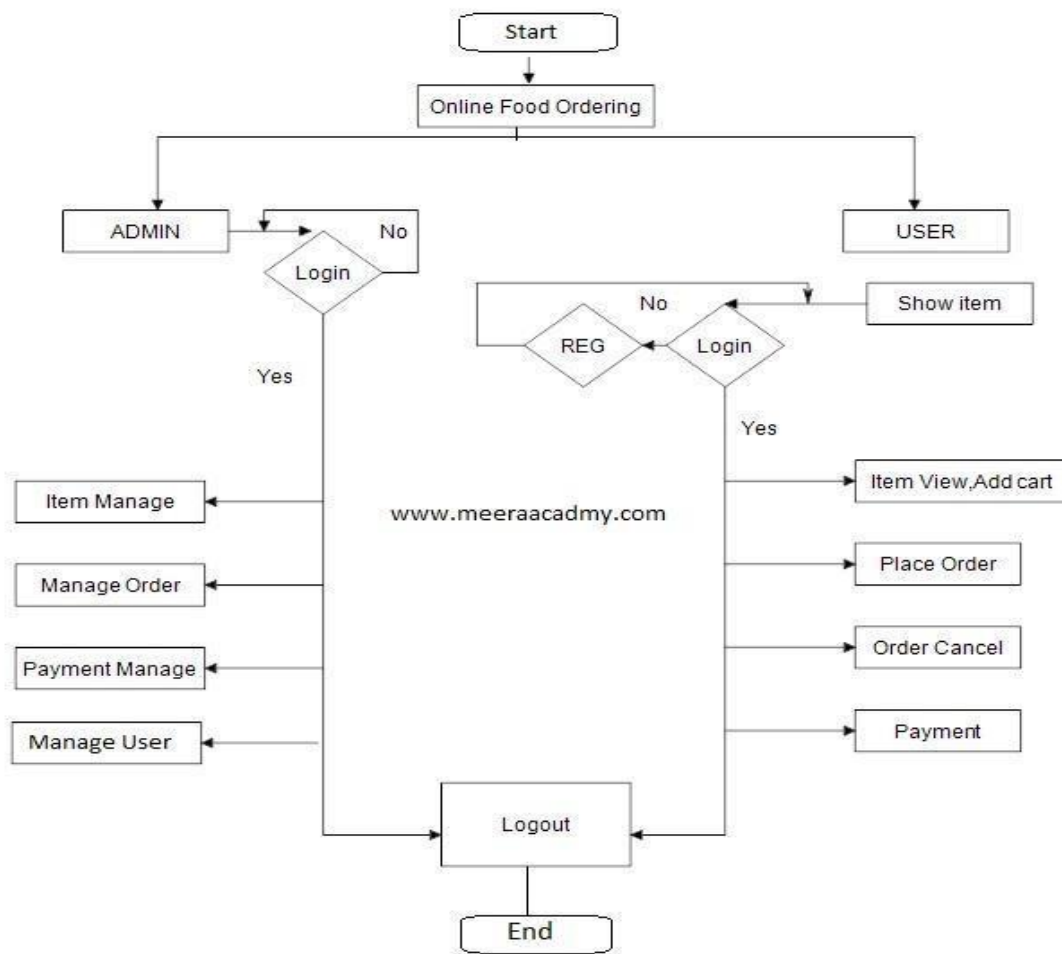


Figure 5.4: System Flowchart of Food Delivery Application

This flowchart represents the workflow of the Food Delivery Web Application developed as part of a learning-based internship. The process begins with user login or registration, followed by browsing restaurants and menu items. The user can add items to the cart, update the cart, and proceed to checkout. After placing the order, the system stores the order details in the database and displays an order confirmation message.

5.5 Choosing Devices

Since this is a web-based application developed as part of a training-based internship, no specialized hardware devices are required. The system can be developed and executed using a standard computer or laptop with basic configuration. The development environment includes a system equipped with a processor (such as Intel i3/i5 or equivalent), a minimum of 4 GB RAM (8 GB recommended for better performance), and sufficient storage space. A stable internet connection is required for downloading tools, accessing documentation, and

testing features when needed. The application is developed using software tools such as Eclipse IDE, Apache Tomcat server, and MySQL database, which run efficiently on standard systems. A web browser like Google Chrome is used to execute and test the application interface, ensuring proper functionality and user interaction. Since the project is intended for learning purposes, this setup allows easy installation, configuration, and execution without the need for advanced infrastructure. The use of commonly available devices ensures that the system is accessible, cost-effective, and suitable for academic environments.

5.6 Designing Units

The application is divided into different functional units to simplify understanding and development. The main units include the user module, restaurant module, menu module, cart module, and order module. Each unit is responsible for handling specific operations within the system. This modular approach helps in understanding how large systems are divided into smaller components, making development and maintenance easier.

5.7 Standards

The project follows basic coding and design standards learned during the internship. The Model–View–Controller (MVC) architecture is used to separate different parts of the application. Standard naming conventions are followed for variables, methods, and classes. Database design follows basic normalization principles to avoid redundancy. Additionally, HTTP protocols are used for communication, and simple security practices such as input validation and PreparedStatement are implemented.

5.8 Mapping with IoTWF Reference Model Layers

Although this project is not an IoT-based system, its structure can be loosely compared to layered models like the IoTWF reference model for learning purposes. The user interface corresponds to the application layer. Servlets represent the processing layer. The MySQL database acts as the data layer, and HTTP communication represents the communication layer. This mapping helps in understanding how layered architectures are used in system design. And this comparison makes it easier to relate web application components to standardized architecture models. It also helps in understanding how different layers interact and communicate with each other in a structured syst

5.9 Domain Model Specification

The domain model defines the key entities involved in the application. These include User, Restaurant, MenuItem, Cart, Order, and OrderItem. A user can register, log in, and place orders. Restaurants contain multiple menu items. The cart temporarily stores selected items before order placement. This model helps in understanding how data is structured and how different entities are related within the system.

5.10 Communication Model

The communication within the application follows the standard HTTP request-response model. GET requests are used for retrieving web pages, while POST requests are used for submitting data such as login credentials and order details. Session management is handled through HTTP sessions, allowing the system to maintain user state across multiple requests. Additionally, AJAX is used for dynamic updates, such as modifying cart quantities without reloading the entire page, thereby improving the overall user experience.

5.11 IoT Deployment Level

This project does not involve IoT deployment. However, it can be considered as a simple web application deployed in a local environment. The system runs on an Apache Tomcat server, and MySQL is used for data storage. The application is accessed through a web browser. This setup helps in understanding how applications are deployed during development.

5.12 Functional View

The functional view of the system describes the operations available to different users. Customers can register, log in, browse restaurants, view menus, manage their cart, and place orders. Administrators, on the other hand, have access to features such as managing menu items and viewing all customer orders. The system also performs internal operations such as validating user credentials, managing sessions, calculating order totals, and ensuring secure access to restricted functionalities.

And the system ensures smooth interaction between different modules to provide a seamless user experience. It also supports efficient handling of user requests and maintains consistency in data processing across the application.

5.13 Mapping IoT Deployment Level with Functional View

Although the system is not IoT-based, the interaction between components can be understood in a layered way. The client handles user input, the server processes requests, and the database stores data. Each functionality involves coordination between these components. This helps in understanding how system layers support application functions.

5.14 Operational View

The operational view explains how the system behaves during execution. When a user performs an action, a request is sent to the server. The server processes the request using Servlets, interacts with the database, and sends a response back to the user. This cycle continues for all user interactions and helps in understanding system operation.

5.15 Database Design

The database for the Food Delivery Web Application has been structured to efficiently manage and store application data using MySQL. Proper normalization techniques have been applied to reduce redundancy and ensure consistency across all data tables. The schema is organized in Third Normal Form (3NF), which helps in maintaining logical relationships between different entities. Each table is uniquely identified using a primary key, while foreign keys are used to establish relationships between related tables. This structured design ensures data integrity, improves performance, and simplifies future maintenance and scalability of the system.

In addition to normalization, careful attention has been given to defining appropriate data types and constraints for each attribute to ensure accuracy and reliability of stored data. Constraints such as NOT NULL and UNIQUE are applied where necessary to prevent invalid or duplicate entries. Indexing techniques can also be incorporated to enhance query performance, especially when handling large volumes of data. The database design supports efficient retrieval and manipulation of information, enabling smooth execution of operations such as user authentication, menu display, cart management, and order processing.

Furthermore, relationships between entities are designed to reflect real-world interactions within the application. For example, a single user can place multiple orders, and each order can contain multiple items, which are managed through separate relational tables. This modular approach not only improves clarity but also makes the system flexible for future enhancements such as adding payment details, delivery tracking, or user reviews. Overall, the

database structure plays a critical role in ensuring that the application functions reliably, securely, and efficiently.

The relationships between different entities in the system are illustrated in Figure 5.4.

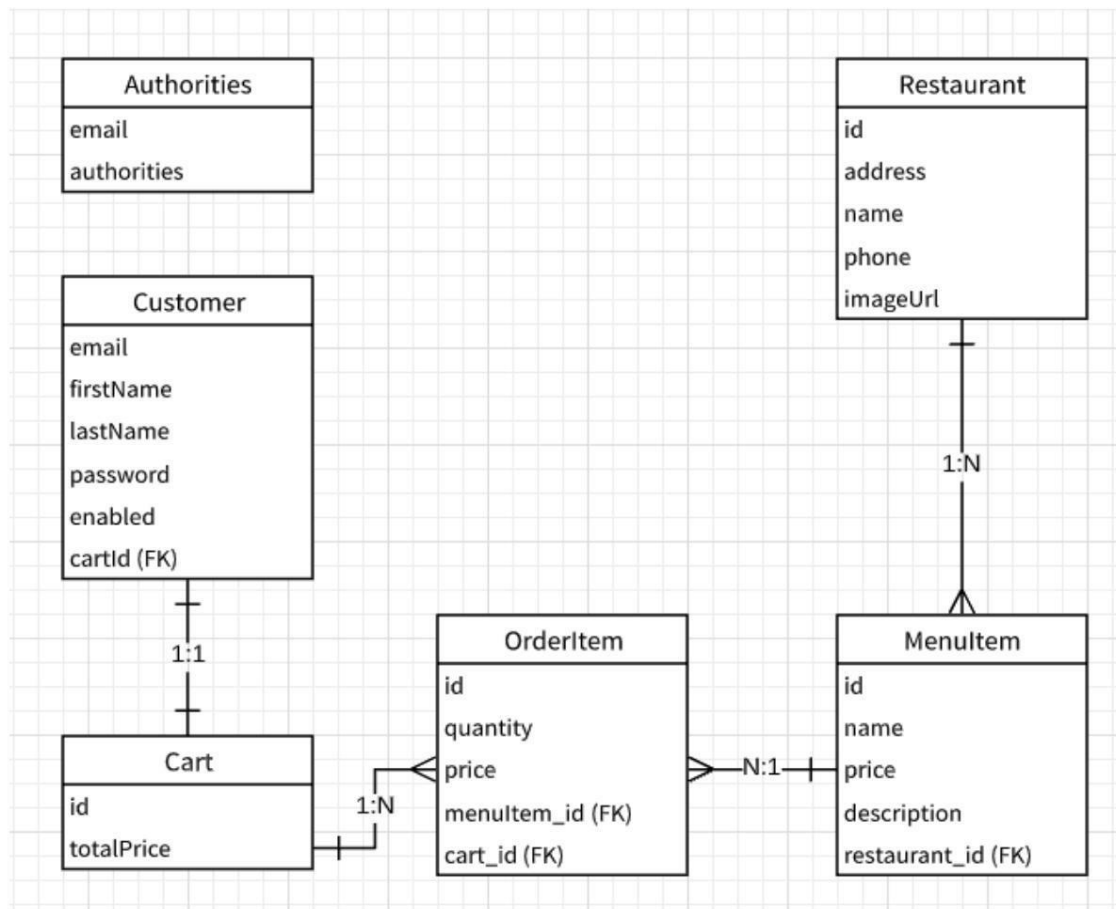


Figure 5.5: Entity Relationship Diagram of Food Delivery System

This ER diagram represents the database structure of the Food Delivery Web Application. It shows relationships between users, restaurants, menu items, and orders. A user can place multiple orders, each order is linked to a restaurant, and each order contains multiple items stored in the order_item table. Menu items are associated with specific restaurants, forming a structured relational database.

Table 5.1: Database Schema Description

Table Name	Primary Key	Key Columns	Purpose
users	user_id (INT, AUTO_INCREMENT)	name, email, password, phone, role	Stores customer and administrator details
restaurants	restaurant_id (INT, AUTO_INCREMENT)	name, cuisine_type, description, image_url	Stores restaurant information
menu_items	item_id (INT, AUTO_INCREMENT)	restaurant_id (FK), name, description, price, image_url	Stores menu items linked to restaurants
cart	cart_id (INT, AUTO_INCREMENT)	user_id (FK), created_at	Represents the active cart of a user
cart_items	cart_item_id (INT, AUTO_INCREMENT)	cart_id (FK), item_id (FK), quantity	Stores items added to the cart with quantity
orders	order_id (INT, AUTO_INCREMENT)	user_id (FK), total_amount, delivery_address, status, order_date	Stores order details placed by users
order_items	order_item_id (INT, AUTO_INCREMENT)	order_id (FK), item_id (FK), quantity, price_at_order	Stores individual items within each order

Table 5.1 presents the detailed structure of the database used in the system. It includes all the tables, their primary keys, important columns, and their respective purposes. This schema forms the backbone of the application and supports all major functionalities such as user management, restaurant browsing, cart handling, and order processing. As shown in Table 5.1, the users table is responsible for storing both customer and administrator information, enabling authentication and role-based access control. The restaurants and menu_items tables manage the food catalogue. Each restaurant can have multiple menu items, which is achieved through a foreign key relationship (restaurant_id), ensuring proper data organization. The cart and cart_items tables handle temporary user selections before placing an order. This separation

allows efficient management of multiple items and quantities within a cart. The orders and order_items tables manage completed transactions. While the orders table stores overall order details, the order_items table captures item-level information, maintaining accuracy and avoiding data redundancy. Overall, the database is designed using relational principles with proper normalization and foreign key constraints, ensuring data integrity, scalability, and efficient query performance.

Chapter 6

HARDWARE, SOFTWARE AND IMPLEMENTATION

6.1 Hardware Requirements

Table 6.1: Hardware Requirements

Component	Minimum Configuration	Recommended Configuration
Processor	Operating System	Intel Core i5 or higher (2.5 GHz and above)
Memory (RAM)	4 GB	8 GB or more
Storage	50 GB HDD	256 GB SSD (for better performance)
Display	1024 × 768 resolution	1920 × 1080 (Full HD)
Network	Basic LAN / Wi-Fi	High-speed LAN (1 Gbps preferred)
Operating System	Windows 10 / Ubuntu 18.04	Windows 10/11 or Ubuntu 20.04 and above

Table 6.1 describes the minimum and recommended hardware configurations required for the development and smooth execution of the application. The system requires at least an Intel Core i5 processor (2.5 GHz or higher) to efficiently handle application processing, server operations, and multitasking activities. In terms of memory, a minimum of 4 GB RAM is necessary, while 8 GB or more is recommended to ensure better performance when running multiple tools such as IDEs, servers, and databases simultaneously. For storage, the system should have at least 50 GB of HDD space; however, a 256 GB SSD is recommended for faster

data access, improved speed, and overall system responsiveness. The display requirement ranges from a minimum resolution of 1024×768 to a recommended resolution of 1920×1080 (Full HD), which provides a better visual experience during development and usage. Additionally, the system should support basic LAN or Wi-Fi connectivity, while a high-speed LAN connection (preferably 1 Gbps) is recommended for efficient data transfer and deployment. The table also specifies operating system compatibility, including Windows 10 or Ubuntu 18.04 as the minimum requirement, with Windows 10/11 or Ubuntu 20.04 and above recommended for better performance, security, and compatibility. Overall, these requirements ensure a stable and efficient environment for developing and running the application.

6.2 Software Requirements

Table 6.2: Software Requirements

Software Tool	Version	Description	License Type
JDK(Java Development Kit)	11 (OpenJDK)	Used for compiling and executing Java programs	Open Source
Eclipse IDE	2023-06	Development environment for coding	Open Source
Apache Tomcat	9.x	Web server and servlet container	Apache License
MySQL Server	8.0	Database management system	Community Edition
MySQL Workbench	8.0	Tool for database design and management	Community Edition
MySQL Connector/J	8.x	Enables Java-MySQL connectivity	GPL
Web Browser	Latest	Used for testing the application	Free
Git	2.x	Version control system	Open Source

Table 6.2 presents the software tools and technologies used for the development, testing, and deployment of the application. It includes details such as tool versions, their purpose, and license types, ensuring that the development environment is properly defined and reproducible. The application is developed using Java, supported by the JDK (Java Development Kit) 11 (OpenJDK), which is used for compiling and executing Java programs. For coding and development, the Eclipse IDE (2023-06) is utilized as an integrated development environment that provides features such as code editing, debugging, and project management. The application is deployed on Apache Tomcat 9.x, which acts as a web server and servlet container for running Java-based web applications. For database management, MySQL Server 8.0 (Community Edition) is used to store and manage application data efficiently.

Additionally, MySQL Workbench 8.0 is used as a graphical tool for database design, querying, and administration. To establish connectivity between the Java application and the MySQL database, MySQL Connector/J (8.x) is used, which enables seamless interaction between the backend and the database. A web browser (latest version) is required for testing and accessing the application interface.

Furthermore, Git (2.x) is used as a version control system to manage source code, track changes, and support collaborative development. Most of these tools are open-source or free to use, making the development process cost-effective and accessible. Overall, these software requirements provide a complete and efficient environment for building, running, and maintaining the application.

6.3 User Module

6.3.1 User Registration and Login Module

```
public int addUser(User user) {  
    String sql = "INSERT INTO users  
(username,password,email,address,phone_number,role,last_login_date,created_date)  
VALUES (?,?,,?,?,,?,?)";  
  
    try (Connection con = DriverManager.getConnection(URL, USER, PASS);  
        PreparedStatement ps = con.prepareStatement(sql)) {  
  
        ps.setString(1, user.getUsername());  
        ps.setString(2, user.getPassword());    ps.setString(3,  
        user.getEmail());  
  
        return ps.executeUpdate();  
    } catch (Exception e) {  
        e.printStackTrace();    }  
    return 0;  
}
```

The `addUser()` method is used to register a new user in the database. It executes an SQL INSERT query using `PreparedStatement`, which ensures secure data handling and prevents SQL injection. The method sets user details such as username, password, and email, and returns the result of the operation.

6.3.2 User Data Retrieval Module

```
public User getUserByEmail(String email)
{
    String sql = "SELECT * FROM users WHERE email=?";

    try (Connection con = DriverManager.getConnection(URL, USER, PASS);
        PreparedStatement ps = con.prepareStatement(sql)) {

        ps.setString(1, email);
        ResultSet rs = ps.executeQuery();
        if (rs.next()) {
            return mapUser(rs);
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
    return null;
}
```

The `getUserByEmail()` method is used to fetch user details from the database based on the provided email address. It prepares and executes an SQL `SELECT` query using `PreparedStatement`, which helps in securely passing the email parameter and prevents SQL injection. The query searches for a matching record in the users table, and the result is stored in a `ResultSet` object. If a matching record is found, the data is extracted and converted into a `User` object using a helper method. This method plays an important role in the login process, as it allows the system to verify whether the entered email exists and retrieve the corresponding user information for authentication.

6.3.3 User Update and Management Module

```
String sql = "UPDATE users SET username=?, email=? WHERE user_id=?";  
PreparedStatement ps = con.prepareStatement(sql); ps.executeUpdate();
```

This code is used to update existing user information in the database. It executes an SQL UPDATE query to modify user details based on the user ID. This allows the system to manage and update user data efficiently.

6.3.4 Order Processing Module

```
public int addOrder(Orders order) {  
  
    String sql = "INSERT INTO orders (user_id, restaurant_id, total_amount,  
delivery_address, payment_method, order_status) VALUES (?, ?, ?, ?, ?, ?)";  
  
    try (Connection con = DriverManager.getConnection(URL, USER, PASS);  
        PreparedStatement ps = con.prepareStatement(sql,  
Statement.RETURN_GENERATED_KEYS)) {  
  
        ps.setInt(1, order.getUserId());  
ps.setInt(2, order.getRestaurantId());  
ps.setDouble(3, order.getTotalAmount());  
  
        ps.executeUpdate();  
  
        ResultSet rs = ps.getGeneratedKeys();  
if (rs.next()) {            return rs.getInt(1);  
        }  
  
    } catch (Exception e) {  
e.printStackTrace();    }  
return 0;  
}
```

The `addOrder()` method is used to store order details in the database. It executes an SQL INSERT query using `PreparedStatement`, which ensures secure and efficient data handling. The method sets important values such as user ID, restaurant ID, and total amount. After inserting the record, it retrieves the generated order ID using `getGeneratedKeys()`, which helps in tracking the created order. This module plays a key role in managing customer orders within the system.

6.4 Integration of Frontend, Backend, and Database

The integration of the frontend, backend, and database layers is achieved through a structured request-response cycle. The frontend, developed using JSP, HTML, CSS, and JavaScript, sends user requests to specific URLs that are mapped to Servlet controllers in the configuration file. The Servlets act as the controller layer, where they process requests, perform business logic, and interact with the database through the DAO layer.

After processing, the Servlets forward the results to JSP pages, which generate the final HTML response displayed to the user. For example, a URL mapping links the path `/login` to the corresponding `LoginServlet`. When a user accesses this URL, the server routes the request to the Servlet, which validates user credentials or displays the login page. Based on the result, it forwards to a JSP page or redirects the user to another page, such as the home screen. For better security, JSP files are stored inside the `WEB-INF/views` directory instead of the `WebContent` folder, preventing direct browser access and ensuring that all navigation is controlled through Servlets.

6.5 Simulation

Simulation is used to test and validate the functionality of the Food Delivery Web Application before deployment. The application was executed on a local server using Apache Tomcat, where different modules such as user registration, login, menu display, cart operations, and order processing were tested under various scenarios.

Different input cases, including both valid and invalid data, were used to evaluate system behavior and error handling. Browser-based testing was carried out using Google Chrome to simulate real user interaction and ensure smooth navigation across different pages. Database operations were verified by checking stored data in MySQL to confirm that all transactions were executed correctly. This simulation process helped identify potential issues and improve system performance, ensuring that the application operates reliably and meets the expected functional requirements.

Chapter 7

EVALUATION AND RESULTS

This chapter presents the testing approach, test cases, test results, and observations obtained during the evaluation phase of the Food Delivery Web Application. The testing process was designed to ensure that the application meets its functional requirements, handles edge cases effectively, and provides a consistent and reliable user experience.

7.1 Test Points

The following major test points were identified based on the functional requirements of the system:

- User registration with both valid and invalid inputs
- User login with correct credentials, incorrect passwords, and non-existent email addresses
- Restaurant and menu browsing to verify that accurate data is displayed
- Cart operations, including adding items, updating quantities, and removing items
- Order placement, including successful orders and attempts to checkout with an empty cart
- Admin login and access control to ensure that customers cannot access admin pages
- Admin menu management, including adding, editing, and deleting menu items
- Session management to verify that logged-out users cannot access protected pages

7.2 Test Plan

The test plan defines the overall strategy used to evaluate the functionality and security of the Food Delivery Web Application. The testing process ensures that all modules operate correctly and meet the specified requirements. The application was tested across key functional areas, including user registration, login, restaurant and menu browsing, cart management, and order processing. Both positive and negative test scenarios were used to validate system behavior and error handling.

Functional testing and user interface testing were performed to ensure proper operation and navigation, while security testing was conducted to prevent unauthorized access and SQL injection. Each test case was designed with specific inputs and expected outputs. The results

were compared with expected outcomes to confirm that the application performs reliably. Additionally, testing helped in identifying and fixing errors during development. It also improved the overall performance and stability of the system.

7.3 Test Case

Table 7.1: Test Cases

TC ID	Module	Test Descriptio	Input	Expected Result	Type
TC-01	Registration	Valid user registration	Name, unique email, password, phone number	Account is created successfully and redirected to login page	Positive
TC-02	Registration	Duplicate email registration	Already registered email	Error message: Email already exists	Negative
TC-03	Registration	Empty input fields	Submit form without entering data	Validation error displayed and form reloaded	Negative
TC-04	Login	Valid login credentials	Correct email and password	Login successful and session created	Positive
TC-05	Login	Incorrect password	Correct email with wrong password	Error message: Invalid credentials	Negative
TC-06	Login	Non-existent email	Email not available in database	Error message: Invalid credentials	Negative
TC-07	Browsing	Load restaurant list	Logged-in user	All restaurants are displayed correctly	Positive

TC-08	Browsing	Load menu items	Select a restaurant	Menu items displayed with correct prices	Positive
TC-09	Cart	Add item to cart	Click “Add to Cart” on item	Item is added to cart successfully	Positive
TC-10	Cart	Update item quantity	Change quantity to 3	Subtotal updates correctly and database is updated	Positive
TC-11	Cart	Remove item from cart	Click “Remove” on item	Item is removed and total is recalculated	Positive
TC-12	Cart	Add duplicate item	Add same item again	Item quantity is increased accordingly	Positive
TC-13	Order	Place order with items	Confirm order with address	Order ID generated and cart is cleared	Positive
TC-14	Order	Checkout with empty cart	Attempt checkout without items	Error message: Cart is empty	Negative
TC-15	Security	Direct JSP access	Enter JSP URL in browser	User is redirected to login page	Positive
TC-16	Security	SQL injection attempt	Enter malicious input in login	Login fails and no SQL error is exposed	Negative
TC-17	Admin	Admin login	Valid admin credentials	Admin dashboard is displayed	Positive
TC-18	Admin	Unauthorized admin access	Customer tries admin URL	Access denied and redirected to home page	Negative

TC-19	Admin	Add menu item	Valid item details	Item is successfully added to menu	Positive
TC-20	Admin	Delete menu item	Delete existing item	Item is removed from database and menu	Positive

Table 7.1 presents the set of test cases designed to validate the functionality and reliability of the application. It includes different modules such as registration, login, and browsing, along with test descriptions, input conditions, expected results, and test types (positive and negative). These test cases ensure that both valid and invalid user interactions are properly handled by the system. For the registration module, test cases verify scenarios such as successful user registration with valid inputs, handling of duplicate email entries, and validation for empty input fields. The system is expected to create an account and redirect the user to the login page in positive cases, while displaying appropriate error messages in negative scenarios.

.In the login module, test cases cover valid login credentials, incorrect password attempts, and login attempts using non-existent email addresses. These tests ensure that authentication is secure and that proper error messages are displayed when invalid credentials are provided. The browsing module includes test cases to verify that logged-in users can view the list of restaurants and access menu items correctly.

These tests confirm that data is fetched and displayed properly from the database. Overall, the test cases in Table 7.1 help ensure that the application behaves as expected under different conditions, improves system reliability, and enhances user experience by handling errors effectively.

7.4 Test Results

Table 7.2: Test Results

TC ID	Status	Actual Result	Notes
TC-01	PASS	User account created and redirected to the login page	
TC-02	PASS	Appropriate error message displayed	
TC-03	PASS	HTML5 validation triggered for required fields	Client-side validation effective
TC-04	PASS	Session created and home page loaded successfully	
TC-05	PASS	Generic error message displayed; no password hints provided	
TC-06	PASS	Generic error message displayed; email details not disclosed	
TC-07	PASS	All test restaurants displayed correctly	
TC-08	PASS	Menu items displayed with accurate pricing	
TC-09	PASS	Item added to cart and displayed immediately	

TC-10	PASS	Subtotal updated dynamically using AJAX; database updated	
TC-11	PASS	Item removed successfully and total recalculated correctly	
TC-12	PASS	Existing cart item quantity]	
TC-13	PASS	Order ID generated and cart cleared after placement	
TC-14	PASS	Redirected with message indicating empty cart	
TC-15	PASS	Unauthorized access prevented; redirected to login page	
TC-16	PASS	SQL injection attempt blocked using PreparedStatement	Logged for monitoring
TC-17	PASS	Admin dashboard loaded successfully with all Functionalities	
TC-18	PASS	Access denied for unauthorized user; redirected to home page	

TC-19	PASS	New menu item added successfully and displayed	
TC-20	PASS	Menu item deleted successfully and page refreshed	

Table 7.2 presents the outcomes of the executed test cases, highlighting the system's behaviour under various scenarios across different modules. It includes the status of each test case, actual results observed during execution, and additional notes where applicable. All test cases from TC-01 to TC-20 have been successfully executed and marked as PASS, indicating that the application meets the expected functional and non-functional requirements.

The results confirm that the registration and login modules are functioning correctly, with proper validation, secure authentication, and appropriate error handling for invalid inputs. Client-side validations (such as HTML5 required fields) and server-side checks are effectively implemented to ensure data integrity and user security. The browsing and cart management functionalities have been thoroughly tested, demonstrating that restaurant listings and menu items are displayed accurately. Cart operations such as adding items, updating quantities, removing items, and dynamic subtotal calculation using AJAX are working as expected, with real-time updates reflected in both the user interface and database. The order processing module has also performed successfully, where orders are placed correctly, unique order IDs are generated, and the cart is cleared after successful transactions.

Edge cases, such as attempting to place an order with an empty cart, are handled properly with appropriate user feedback. Security-related test cases show that unauthorized access is prevented, and sensitive operations are protected. Additionally, SQL injection attempts are successfully blocked using prepared statements, ensuring database security. The admin functionalities have been verified, including dashboard access, adding new menu items, and deleting existing items. Access control mechanisms ensure that only authorized users can perform administrative actions. Overall, the test results demonstrate that the system is stable, secure, and performs reliably across all major functionalities, making it ready for deployment and real-world usage.

7.5 Insights and Observations

The testing phase provided valuable insights into the application's behavior and the effectiveness of the design decisions implemented during development. The use of `PreparedStatement` across all DAO classes proved to be highly effective in preventing SQL injection attacks, as demonstrated in test case TC-16. The injected input was treated as a normal parameter value rather than executable SQL, resulting in a failed login attempt instead of unauthorized access. This confirms the importance of following secure coding practices.

Session management implemented through the Servlet container functioned reliably. The use of Servlet filters ensured that unauthenticated users were redirected to the login page when attempting to access protected resources.

This mechanism guarantees that sensitive functionalities remain inaccessible without proper authentication. The AJAX-based cart update feature performed efficiently under normal conditions. However, it was observed that without proper server-side validation, invalid inputs such as negative quantities could be submitted using browser developer tools. To address this issue, additional validation checks were implemented on the server side. The order processing module, which uses JDBC transactions, demonstrated strong reliability. During testing, artificial errors were introduced to verify rollback functionality.

Chapter 8

SOCIAL, LEGAL, ETHICAL, SUSTAINABILITY, AND SAFETY ASPECTS

8.1 Social Aspects

The Food Delivery Web Application developed in this project is part of a training-based learning process aimed at understanding how real-world systems are designed and implemented. The application demonstrates how online food delivery platforms improve convenience by allowing users to browse restaurants and place orders digitally. It also highlights how such systems can support small businesses by increasing their visibility and reach. Through this project, the social impact of digital platforms in enhancing user experience and supporting local businesses has been studied and understood.

8.2 Legal Aspects

This project is developed for educational purposes, and therefore does not involve real user data or live deployment. However, it helps in understanding the legal responsibilities associated with handling user information in real-world applications. Concepts such as data privacy, user consent, and secure data storage have been considered during development. The system design allows for future implementation of legal requirements such as data protection policies, encryption, and secure communication if deployed in a real environment.

8.3 Ethical Aspects

The application has been developed by following basic ethical principles in software development. Only necessary user data fields are included, and no unnecessary data collection mechanisms are implemented. The system maintains transparency by clearly displaying information such as menu items, prices, and order details. Since this is a training project, the focus is on learning ethical coding practices, including writing secure and maintainable code, rather than deploying a commercial system.

8.4 Sustainability Aspects

The project emphasizes sustainability from a technical learning perspective. It demonstrates the use of efficient and widely used technologies such as Java, MySQL, and Apache Tomcat. The system is designed using structured architecture patterns like MVC and DAO, which support maintainability and scalability in real-world applications. Additionally, the use of open-source tools reduces cost and promotes long-term sustainability. The project helps in understanding how scalable and maintainable systems are built.

8.5 Safety Aspects

Although this is a prototype developed for learning purposes, basic safety and security measures have been implemented. The use of PreparedStatement helps prevent SQL injection attacks, and session management ensures that only authenticated users can access certain functionalities. Input validation is applied to handle incorrect or invalid data. The project also demonstrates the importance of transaction management in maintaining data consistency. These safety practices provide a foundational understanding of secure application development.

Chapter 9

CONCLUSION

This project focused on understanding the core concepts of a Food Delivery Web Application as part of a training-based internship at Tap Academy. The system demonstrates the basic workflow of an online food ordering platform, including user registration, login, restaurant browsing, menu viewing, cart management, and order placement, along with an administrative module to manage menu items and monitor customer orders. The primary objective of this project was to gain practical exposure to full-stack web development rather than building a production-level system. The application was developed using Java Servlets, JSP, MySQL, and Apache Tomcat, following the Model–View–Controller (MVC) architecture, while the Data Access Object (DAO) pattern was used for structured database operations. Through this implementation, a clear understanding was gained of how frontend, backend, and database components interact within a web application. The project was developed in a step-by-step manner and tested using multiple test cases, including both positive and negative scenarios, to verify system functionality and behaviour.

This internship provided valuable hands-on experience in key areas such as session management, JDBC connectivity, database design, and structured application development. It also enhanced problem-solving skills and improved the ability to debug and handle errors effectively. Additionally, the project emphasized the importance of secure coding practices, such as using Prepared Statement to prevent SQL injection and validating user inputs. Overall, the project helped bridge the gap between theoretical knowledge and practical implementation. Although the application successfully demonstrates fundamental concepts, it remains a learning prototype. Future enhancements may include integrating a payment gateway, implementing password encryption techniques like hashing, enabling real-time order tracking, improving the user interface, and extending support to mobile platforms. These improvements would help transform the application into a more complete and production-ready system.

REFERENCES

The following references were used to understand the concepts and technologies implemented in this project:

1. Tap Academy. Full Stack Java Training Materials and Learning Platform.
2. Oracle Corporation. Java Servlet and JSP Documentation. Available at: <https://docs.oracle.com/javase/>
3. Apache Software Foundation. Apache Tomcat Documentation. Available at: <https://tomcat.apache.org>
4. Oracle Corporation. MySQL 8.0 Reference Manual. Available at: <https://dev.mysql.com/doc>
5. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
6. Core J2EE Patterns. Prentice Hall.
7. JavaServer Pages (JSP) Standard Tag Library. O'Reilly Media.
8. W3Schools. Web Development Tutorials (HTML, CSS, JavaScript, JDBC). Available at: <https://www.w3schools.com>
9. GeeksforGeeks. Java and Web Development Concepts. Available at: <https://www.geeksforgeeks.org>
10. Eclipse Foundation. Eclipse IDE Documentation. Available at: <https://www.eclipse.org>
11. Thrupthidr. Food Delivery Web Application. GitHub, 2026. Available at: <https://github.com/Thrupthidr/Food-Delivery-Web-Application.git>

APPENDIX

This appendix section provides additional supporting details related to the Food Delivery Web Application developed during the internship. It includes sample code snippets, database structure, and application screenshots to enhance understanding of the system implementation. The project was developed using Java Servlets, JSP, MySQL, and Apache Tomcat, following the Model–View–Controller (MVC) architecture. This section highlights key components that demonstrate how different modules of the application are implemented and integrated.

The included code snippets represent important functionalities such as user registration, login validation, and order processing. These examples illustrate how database operations are performed using JDBC and how secure coding practices, such as the use of PreparedStatement, help prevent SQL injection and maintain data integrity. In addition, this section includes screenshots of various application pages, such as the login page, restaurant listing, menu display, cart management, and admin panel. These visuals provide a clear understanding of the user interface and the overall workflow of the system. The database structure presented here explains how data is organized across multiple related tables, including users, restaurants, menu items, cart, and orders. This helps in understanding how data is stored, retrieved, and managed within the application.

The following materials are included in this section:

- Sample source code for key modules such as user registration, login, and order processing
- Database schema and table structure used in the application
- Screenshots of application interfaces including login, restaurant listing, menu display, cart, and admin panel

These details are provided for reference and support the explanations given in earlier chapters, offering better clarity on the practical implementation of the system.

A. Source Code

A.1 User Registration Code

```
// User Registration Method public
int addUser(User user) {

    int result = 0;
    String sql = "INSERT INTO users
(username,password,email,address,phone_number,role,last_login_date,created_date)
VALUES (?, ?, ?, ?, ?, ?, ?, ?)";

    try (Connection con = DriverManager.getConnection(URL, USER, PASS);
        PreparedStatement ps = con.prepareStatement(sql)) {

        Timestamp ts = new Timestamp (System.currentTimeMillis());
        ps.setString(1, user.getUsername());    ps.setString(2,
user.getPassword());    ps.setString(3, user.getEmail());
        ps.setString(4, user.getAddress());    ps.setString(5,
user.getPhoneNumber());    ps.setString(6, user.getRole());
        ps.setTimestamp(7, ts);    ps.setTimestamp(8, ts);

        result = ps.executeUpdate();
    } catch (Exception e) {
        e.printStackTrace();    }
    return result;
}
```

This method retrieves user details from the database based on email and is used for login validation.

A.2 User Login Code

// Fetch User by Email (Login)

```
public User getUserByEmail(String email) {  
  
    User user = null;  
    String sql = "SELECT * FROM users WHERE email=?";  
  
    try (Connection con = DriverManager.getConnection(URL, USER, PASS);  
        PreparedStatement ps = con.prepareStatement(sql)) {  
  
        ps.setString(1, email);  
        ResultSet rs = ps.executeQuery();  
        if (rs.next()) {  
            user = mapUser(rs);  
        }  
  
    } catch (Exception e) {  
        e.printStackTrace();    }  
    return user;  
}
```

This method retrieves user details from the database based on email and is used for login validation.

A.3 Order Processing Code

```
// Add Order Method public int
addOrder(Orders order) {    int
orderId = 0;

    String sql = "INSERT INTO orders (user_id, restaurant_id, total_amount,
delivery_address, payment_method, order_status) VALUES (?, ?, ?, ?, ?, ?)";

    try (Connection con = DriverManager.getConnection(URL, USER, PASS);
        PreparedStatement ps = con.prepareStatement(sql,
Statement.RETURN_GENERATED_KEYS)) {

        ps.setInt(1, order.getUserId());
ps.setInt(2, order.getRestaurantId());
ps.setDouble(3, order.getTotalAmount());
ps.setString(4, order.getDeliveryAddress());
ps.setString(5, order.getPaymentMethod());
ps.setString(6, order.getOrderStatus());
ps.executeUpdate();

        ResultSet rs = ps.getGeneratedKeys();
if (rs.next()) {            orderId =
rs.getInt(1);
        }

    } catch (Exception e) {
e.printStackTrace();    }
return orderId;
}
```

This method is used to store order details in the database and generate a unique order ID for each order.

B. Database Schema and Table Structure

The Food Delivery Web Application uses a MySQL database named `food_delivery_app`. The database consists of multiple tables designed to store and manage application data efficiently. Each table represents a specific entity such as users, restaurants, menu items, and orders.

Table B.1: Users Table Structure

Column Name	Data Type	Description
<code>user_id</code>	INT (PK, AUTO_INCREMENT)	Unique user ID
<code>username</code>	VARCHAR(50)	Username of the user
<code>password</code>	VARCHAR(50)	User password
<code>email</code>	VARCHAR(100)	Email address
<code>address</code>	VARCHAR(100)	User address
<code>phone_number</code>	VARCHAR(15)	Contact number
<code>role</code>	VARCHAR(30)	User role (Admin/User)
<code>last_login_date</code>	TIMESTAMP	Last login time
<code>created_date</code>	TIMESTAMP	Account creation date

The users table stores all registered users of the application, including both customers and administrators. Each user is uniquely identified using the `user_id`, which is auto-generated. The `role` field is used to distinguish between normal users and admin users, enabling rolebased access control within the system. Contact details such as `email` and `phone_number` are stored for communication purposes. Additionally, timestamps like `created_date` and `last_login_date` help track user activity and improve system monitoring.

Table B.2: Restaurant Table Structure

Column Name	Data Type	Description
restaurant_id	INT (PK, AUTO_INCREMENT)	Unique restaurant ID
restaurant_name	VARCHAR(150)	Name of restaurant
admin_user_id	INT (FK)	Admin managing restaurant
address	VARCHAR(255)	Restaurant location
cuisine_type	VARCHAR(100)	Type of cuisine
rating	DECIMAL(2,1)	Restaurant rating
eta	INT	Estimated delivery time
image_url	VARCHAR(255)	Restaurant image
is_available	TINYINT(1)	Availability status

The restaurant table maintains detailed information about all restaurants available in the application. Each restaurant is uniquely identified by `restaurant_id` and can be managed by an administrator through the `admin_user_id` field. Attributes such as `cuisine_type`, `rating`, and `eta` (estimated delivery time) enhance the user experience by providing useful filtering and selection options. The `is_available` field indicates whether a restaurant is currently active, ensuring only available restaurants are shown to users.

Table B.3: Menu Table Structure

Column Name	Data Type	Description
menu_id	INT (PK, AUTO_INCREMENT)	Unique menu ID
restaurant_id	INT (FK)	Linked restaurant
menu_name	VARCHAR(150)	Item name
description	VARCHAR(255)	Item description
price	DECIMAL(10,2)	Item price
rating	DECIMAL(2,1)	Item rating
image_url	VARCHAR(255)	Item image
dish_type	VARCHAR(20)	Veg/Non-Veg
is_available	TINYINT(1)	Availability

The menu table stores all food items offered by different restaurants. Each menu item is linked to a specific restaurant using the restaurant_id foreign key, ensuring proper organization of data. Important attributes such as price, rating, and dish_type (veg or non-veg) help users make better decisions while ordering. The is_available field allows dynamic control over item availability, enabling restaurants to update their menu in real time.

Table B.4: Orders Table Structure

Column Name	Data Type	Description
order_id	INT (PK, AUTO_INCREMENT)	Unique order ID
user_id	INT (FK)	User who placed order
restaurant_id	INT (FK)	Restaurant ID
total_amount	DECIMAL(10,2)	Total order price
order_date	TIMESTAMP	Order date
delivery_address	VARCHAR(255)	Delivery address
payment_method	VARCHAR(50)	Payment type
order_status	VARCHAR(50)	Order status

The orders table records all orders placed by users in the system. Each order is associated with a specific user and restaurant through foreign keys, enabling efficient tracking of transactions. The total_amount field represents the overall cost of the order, while delivery_address and payment_method store essential delivery and payment information. The order_status field helps in tracking the progress of the order, such as pending, confirmed, or delivered.

Table B.5: Order Item Table Structure

Column Name	Data Type	Description
order_item_id	INT (PK, AUTO_INCREMENT)	Unique ID
order_id	INT (FK)	Reference to order
menu_id	INT (FK)	Menu item
quantity	INT	Quantity ordered
item_price	DECIMAL(10,2)	Price at order time

The order_item table stores the individual items included in each order, allowing a single order to contain multiple products. Each record links a menu item to an order using foreign keys, ensuring proper relationship management. The quantity field specifies how many units of an item were ordered, while item_price captures the price at the time of purchase. This design helps maintain accurate historical data even if menu prices change later.

C. Application Screenshots

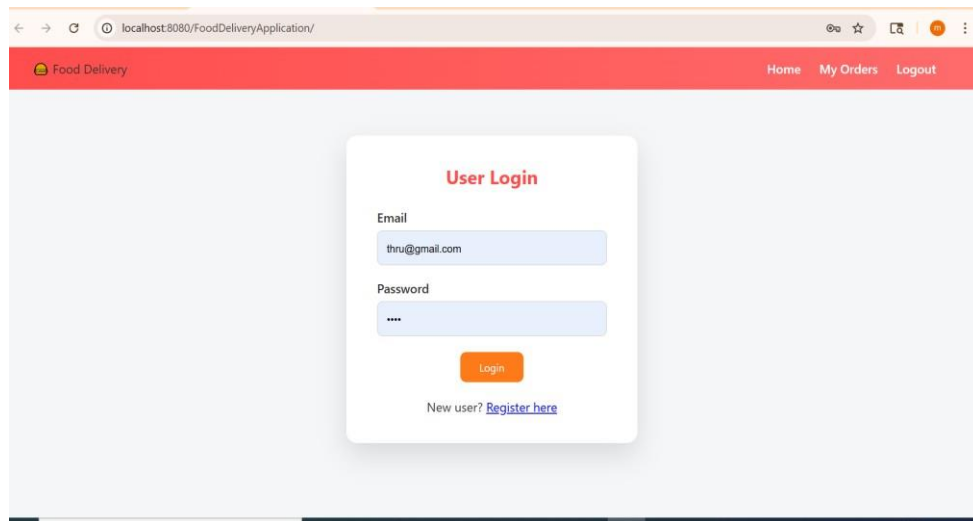


Figure C.1: User Login Page

This screen allows users to log in by entering their registered email and password. The system validates the credentials and redirects the user to the home page upon successful login.



Figure C.2: User Registration Page

This page allows new users to create an account by providing details such as username, email, password, and phone number.

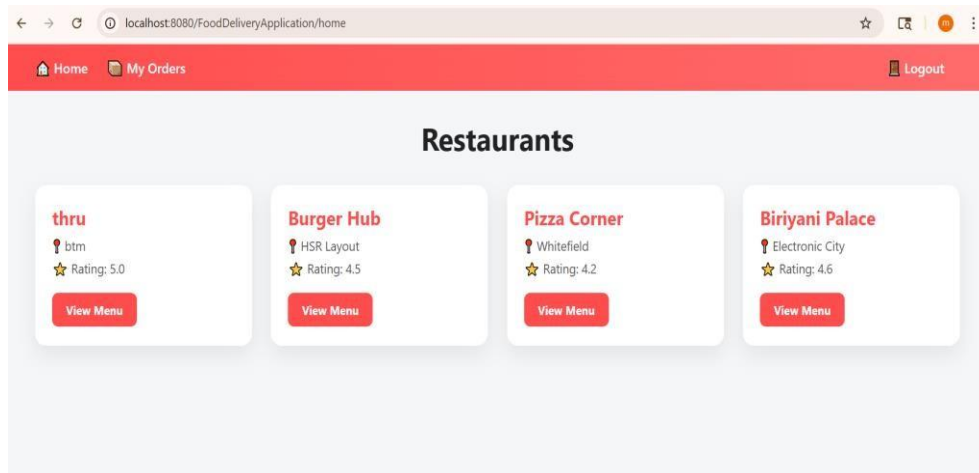


Figure C.3: Restaurant Listing Page

This page displays all available restaurants with details such as name, cuisine type, rating, and estimated delivery time.

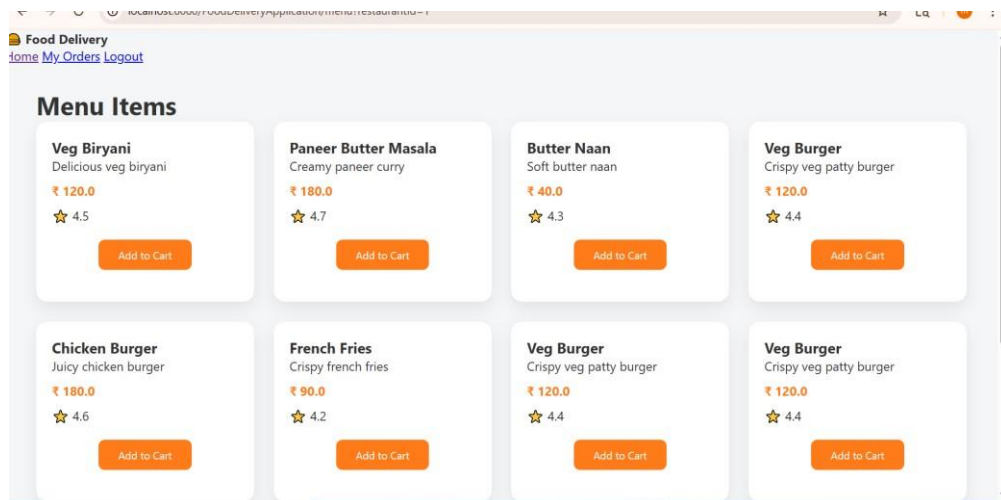


Figure C.4: Menu Page

This page shows menu items of a selected restaurant, including item name, price, description, and availability.

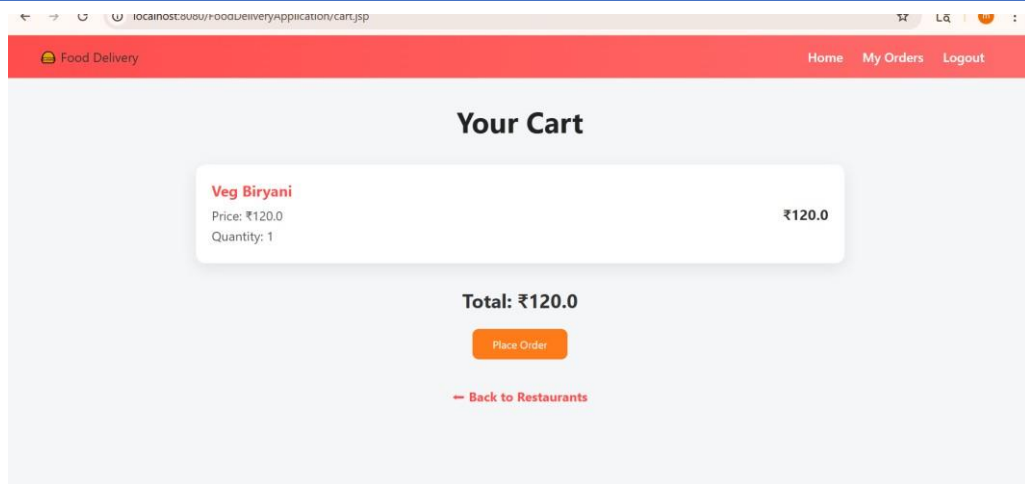


Figure C.5: Cart Page

This page displays items added to the cart, allowing users to update quantities or remove items before placing an order.

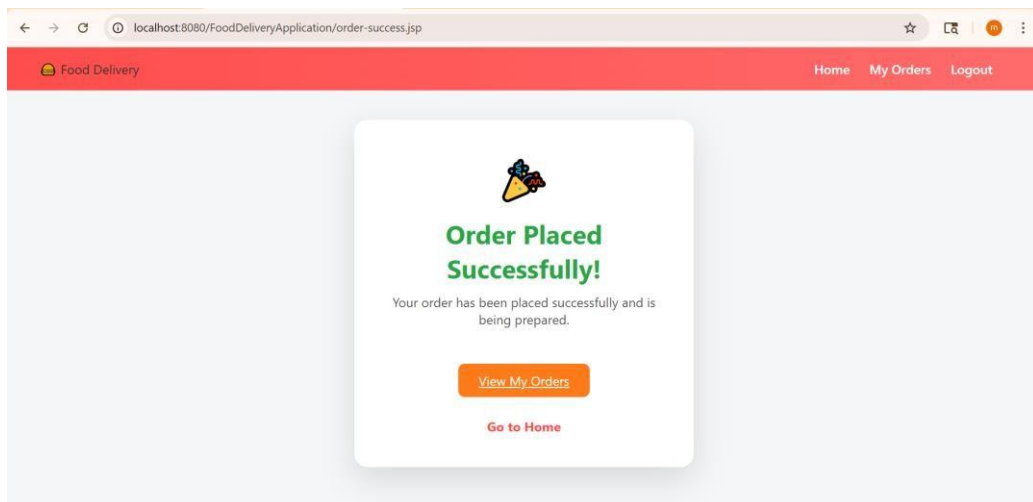


Figure C.6: Order Confirmation Page

This page confirms that the order has been successfully placed and displays order details.